

Классы и объекты (ООП)

https://netology.ru/profile/program/bhembd-25-pdl-1/lessons/523560/lesson_items/2836825

Цель

Закрепить навыки работы с классами в Python

Задача «Модель магазина»

Создайте модель для онлайн-магазина, который управляет товарами и заказами. Ваша задача — реализовать классы для товаров, заказов и магазина, который будет обрабатывать заказы.

1. Класс Product

Атрибуты:

- `name` (строка) — название товара;
- `price` (число) — цена товара;
- `stock` (целое число) — количество товара на складе.

Методы:

- `update_stock(quantity)` — метод, который обновляет количество товара на складе. Если количество становится отрицательным, должно выдаваться сообщение об ошибке.

```
In [1]: class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

    def update_stock(self, quantity):
        """Обновляет количество товара на складе."""
        if self.stock + quantity < 0:
            print(f"Ошибка: недостаточно товара '{self.name}' на складе.")
        else:
            self.stock += quantity
            print(f"Количество товара '{self.name}' обновлено: {self.stock}")

    def __repr__(self):
        return f"{self.name} — {self.price} руб. (в наличии: {self.stock})"
```

2. Класс Order

Атрибуты:

- `products` (словарь) — словарь, где ключом является объект `Product`, а значением — количество этого товара в заказе.

Методы:

- `add_product(product, quantity)` — метод для добавления товара в заказ. Если товара недостаточно на складе, должно выдаваться сообщение об ошибке;
- `calculate_total()` — метод для расчёта общей стоимости заказа.

Дополнительно:

1. Реализуйте возможность удаления товаров из заказа. Добавьте метод `remove_product(product, quantity)` в класс `Order`, который будет удалять указанное количество товара из заказа. Если количество товара в заказе становится равным нулю, удалите товар из словаря `products`.
2. Добавьте функциональность для обработки возвратов товаров и обновления запасов. Реализуйте метод `return_product(product, quantity)` в классе `Order`, который будет возвращать указанный товар обратно в магазин. Этот метод должен: уменьшать количество товара в заказе; вызывать метод `update_stock(quantity)` у объекта `Product`, чтобы увеличить количество товара на складе; если количество товара в заказе становится равным нулю, удалите товар из словаря `products`.

```
In [2]: class Order:
    def __init__(self):
        self.products = {}

    def add_product(self, product, quantity):
```

```

        """Добавляет товар в заказ, если он есть в наличии."""
        if product.stock < quantity:
            print(f"Ошибка: недостаточно товара '{product.name}' на складе.")
            return

        self.products[product] = self.products.get(product, 0) + quantity
        product.update_stock(-quantity)
        print(f"Добавлено {quantity} ед. товара '{product.name}' в заказ.")

    def remove_product(self, product, quantity):
        """Удаляет указанное количество товара из заказа."""
        if product not in self.products:
            print(f"Товар '{product.name}' отсутствует в заказе.")
            return

        if quantity >= self.products[product]:
            del self.products[product]
            print(f"Товар '{product.name}' удалён из заказа.")
        else:
            self.products[product] -= quantity
            print(f"Количество '{product.name}' уменьшено на {quantity} ед.")

        product.update_stock(quantity)

    def return_product(self, product, quantity):
        """Возвращает товар обратно в магазин."""
        if product not in self.products:
            print(f"Товар '{product.name}' отсутствует в заказе.")
            return

        if quantity >= self.products[product]:
            quantity = self.products[product]
            del self.products[product]
            print(f"Все единицы товара '{product.name}' возвращены в магазин.")
        else:
            self.products[product] -= quantity
            print(f"Возвращено {quantity} ед. товара '{product.name}'.")

        product.update_stock(quantity)

    def calculate_total(self):
        """Рассчитывает общую стоимость заказа."""
        total = sum(p.price * q for p, q in self.products.items())
        print(f"Общая стоимость заказа: {total} руб.")
        return total

```

3. Класс Store

Атрибуты:

- `products` (список) — список всех доступных товаров в магазине.

Методы:

- `add_product(product)` — метод для добавления товара в магазин;

- `list_products()` — метод для отображения всех товаров в магазине с их ценами и количеством на складе;
- `create_order()` — метод для создания нового заказа.

Сверьтесь с примером использования.

```
In [3]: class Store:
        def __init__(self):
            self.products = []

        def add_product(self, product):
            """Добавляет товар в магазин."""
            self.products.append(product)
            print(f"Добавлен товар: {product.name}")

        def list_products(self):
            """Выводит список всех товаров в магазине."""
            print("\nСписок товаров в магазине:")
            for product in self.products:
                print(f" - {product}")

        def create_order(self):
            """Создает новый заказ."""
            print("Создан новый заказ.")
            return Order()
```

Проверка

```
In [4]: # Создание магазина и товаров
store = Store()

laptop = Product("Ноутбук", 75000, 5)
phone = Product("Смартфон", 45000, 10)
headphones = Product("Наушники", 5000, 15)

store.add_product(laptop)
store.add_product(phone)
store.add_product(headphones)

# Проверяем, что товары добавлены
assert len(store.products) == 3

# Создание и работа с заказом
order = store.create_order()
order.add_product(laptop, 1)
order.add_product(phone, 2)

# Проверка остатков после заказа
assert laptop.stock == 4, "Количество ноутбуков должно уменьшиться до 4"
assert phone.stock == 8, "Количество смартфонов должно уменьшиться до 8"

# Проверка стоимости заказа
total = order.calculate_total()
```

```

assert total == 75000 * 1 + 45000 * 2, "Неверная сумма заказа"

# Удаляем один телефон из заказа
order.remove_product(phone, 1)
assert phone.stock == 9, "После удаления товара из заказа он должен вернуться на ск
assert order.calculate_total() == 75000 * 1 + 45000 * 1

# Возвращаем ноутбук
order.return_product(laptop, 1)
assert laptop.stock == 5, "После возврата ноутбук должен вернуться на склад"
assert laptop not in order.products, "Ноутбук должен быть удалён из заказа после во

```

Добавлен товар: Ноутбук
 Добавлен товар: Смартфон
 Добавлен товар: Наушники
 Создан новый заказ.
 Количество товара 'Ноутбук' обновлено: 4
 Добавлено 1 ед. товара 'Ноутбук' в заказ.
 Количество товара 'Смартфон' обновлено: 8
 Добавлено 2 ед. товара 'Смартфон' в заказ.
 Общая стоимость заказа: 165000 руб.
 Количество 'Смартфон' уменьшено на 1 ед.
 Количество товара 'Смартфон' обновлено: 9
 Общая стоимость заказа: 120000 руб.
 Все единицы товара 'Ноутбук' возвращены в магазин.
 Количество товара 'Ноутбук' обновлено: 5

Пример использования

```

In [5]: # Создаем магазин
store = Store()

# Создаем товары
product1 = Product("Ноутбук", 1000, 5)
product2 = Product("Смартфон", 500, 10)

# Добавляем товары в магазин
store.add_product(product1)
store.add_product(product2)

# Список всех товаров
store.list_products()

# Создаем заказ
order = store.create_order()

# Добавляем товары в заказ
order.add_product(product1, 2)
order.add_product(product2, 3)

# Выводим общую стоимость заказа
total = order.calculate_total()
print(f"Общая стоимость заказа: {total}")

```

```
# Проверяем остатки на складе после заказа
store.list_products()
```

Добавлен товар: Ноутбук

Добавлен товар: Смартфон

Список товаров в магазине:

- Ноутбук – 1000 руб. (в наличии: 5)
- Смартфон – 500 руб. (в наличии: 10)

Создан новый заказ.

Количество товара 'Ноутбук' обновлено: 3

Добавлено 2 ед. товара 'Ноутбук' в заказ.

Количество товара 'Смартфон' обновлено: 7

Добавлено 3 ед. товара 'Смартфон' в заказ.

Общая стоимость заказа: 3500 руб.

Общая стоимость заказа: 3500

Список товаров в магазине:

- Ноутбук – 1000 руб. (в наличии: 3)
- Смартфон – 500 руб. (в наличии: 7)